23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : B10
Team Members :

Roll No	Student Name
AM.SC.U4CSE25167	MYDHILY MS
AM.SC.U4CSE25119	GOUTHAM KARIYADAN
AM.SC.U4CSE25159	YADHU
AM.SC.U4CSE25168	MEERA

Project Title : MedicareHMS

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	2
2	Introduction	3
3	Requirements (Functional & Non-functional)	4
4	UI Design	5-7
5	Database Design	8-17
6	Database Selection & Justification	17
7	Implementation	18-20
8	Testing & Validation	21
9	Results & Screenshots	22
10	Challenges faced & solutions	23
11	Conclusion & future enhancements	23
	References	24

1. Abstract

The Hospital Management System is designed to convert paper-based records maintained by a multibranch, multispecialty hospital into organized digital records. Separate registers for patients, appointments, admissions, prescriptions, pharmacy stock, laboratory results, surgery, billing and payroll are brought together in a relational database. The project uses PostgreSQL, a web interface and an application server to support record creation, searching, updating, deletion and reporting. Primary keys, foreign keys, composite keys and validation rules help maintain consistent relationships and prevent invalid entries. Raw CSV records are cleaned and organized into related tables, while joined views make information from different hospital activities accessible through one interface. The expected outcome is effective use of digital tools to improve record availability, simplify updates and analysis, and increase the efficiency of hospital administration.

2. Introduction

Background and need

A multispecialty hospital handles several connected activities during one patient visit. Registration identifies the patient, a department assigns a doctor, an appointment reserves time, and treatment may lead to tests, medicines, admission or billing. In a hospital with several branches, these activities must remain connected even when the patient receives services at different locations. Paper registers make each activity easy to record locally, but provide no reliable shared view of the complete process.

Lessons from documented incidents

The UK National Audit Office investigated a backlog of clinical correspondence at NHS Shared Business Services in 2017. Misdirected correspondence was left unprocessed instead of reaching the appropriate practices. The investigation described potential clinical risk, although it had not identified patient harm by its reporting date. The case illustrates how physical correspondence and weak tracking can leave important clinical information unavailable when needed.

Limitations of the existing paper-based approach

- The same patient and service details are copied into several registers, increasing the chance of inconsistent updates.
- Finding an earlier admission, prescription or bill requires manual searching and coordination between branches.
- Separate appointment, room and theatre records make scheduling conflicts harder to identify.
- Stock verification, branch summaries and financial reports require repeated manual consolidation.

How the proposed system addresses the need

The project assigns stable identifiers to records and links them through foreign keys. Shared patient records support branch-level operations; appointments connect patients, doctors and branches; admission records preserve room-allocation history. Prescription details connect medicines with treatment, and pharmacy inventory keeps stock specific to each pharmacy. Searchable registers, joined displays and year-based reports make stored information easier to retrieve and compare.

The project demonstrates these database and administrative workflows as an academic prototype. Its expected efficiency benefits are qualitative: quicker access to organized records, fewer repeated descriptive details, and clearer visibility of hospital activity. No measured clinical benefit or percentage improvement is claimed.

3. Requirements (Functional & Non-functional)

Functional requirements

- Authentication: allow registered users to sign in and restrict API operations according to their assigned role.
- Record management: create, read, search, update and delete permitted records, while protecting data referenced by other tables.
- Hospital organization: maintain branches, departments, doctors, nurses, staff, rooms and equipment.
- Appointments and care: connect each consultation to a patient, doctor and branch; check doctor availability and retain admissions and care assignments.
- Clinical services: maintain prescriptions and medicine details, laboratory results, operation theatres and procedure schedules.
- Pharmacy and billing: maintain pharmacy-specific stock, prices and reorder levels; record appointment bills and payment status.
- Payroll records: maintain pay structures, employee shift logs and salaries with links to applicable completed shifts.
- Reports: display year-based summaries and branch activity; export monthly report data as CSV.

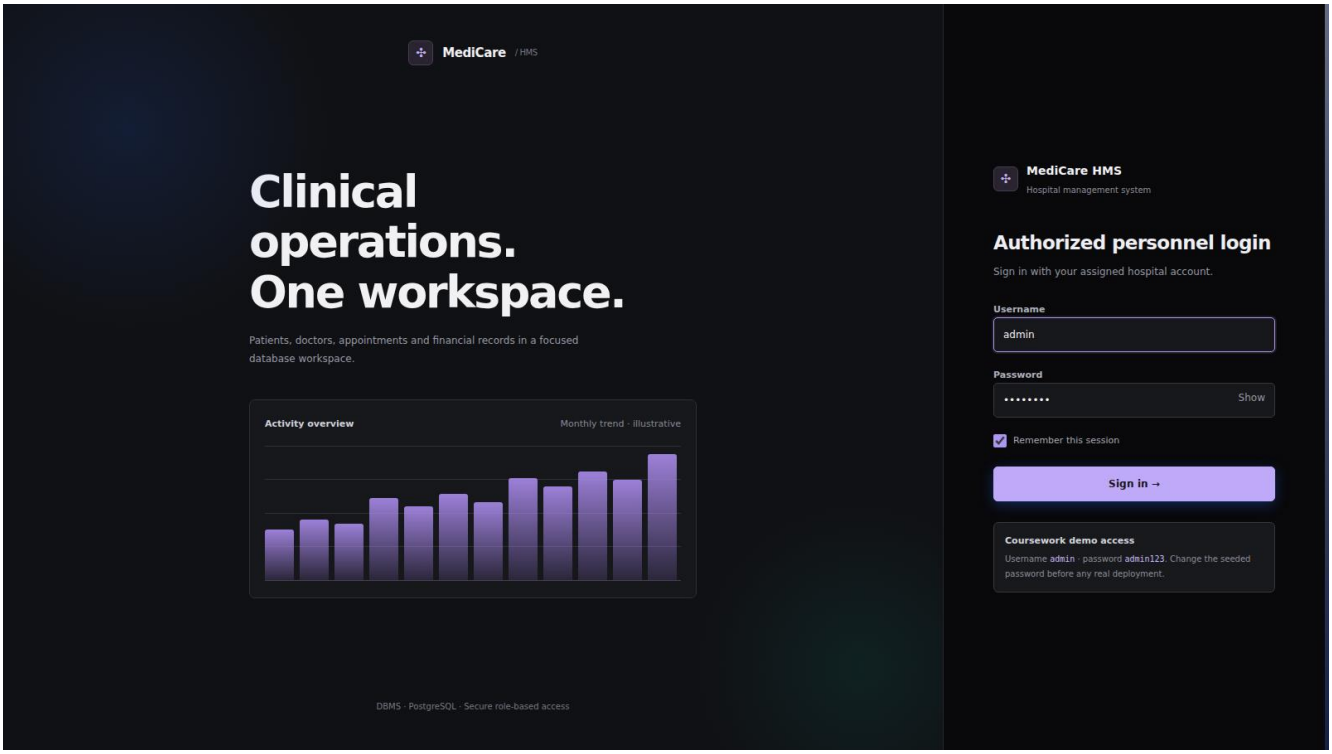
Non-functional requirements

- Data integrity: enforce unique identities, valid foreign keys, allowed values and sensible quantities, dates and time ranges.
- Reliability: use transactions for related changes so that a failed operation does not leave a partly completed update.
- Security: store password hashes, authenticate requests and use parameterized SQL for user-supplied values.
- Usability: provide consistent navigation, labelled forms, search controls, confirmation before deletion and understandable validation messages.
- Performance and growth: use indexed lookup fields and normalized tables to support increasing records and additional branches. Production load targets are not established by this project.
- Maintainability: keep the interface, API logic and database definitions separate, with SQL files documenting the schema.

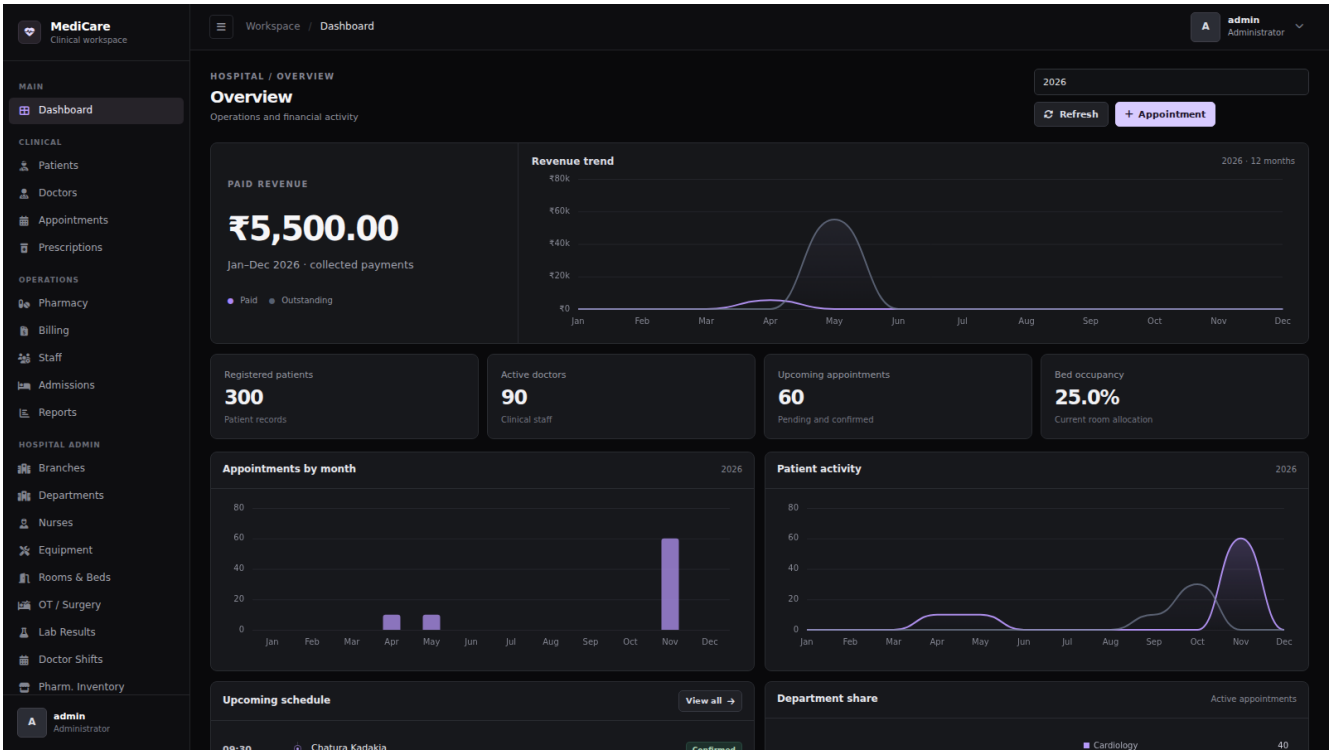
Scope: this version is a coursework application for hospital record management. Payment receipts, a patient mobile app, deployment automation and production security controls remain future work.

4. UI Design

Navigation: Login -> Dashboard -> Module register -> Search / Add / Edit / Delete. Reports -> Select year -> Export CSV. Report and pharmacy screens are shown in Section 9.

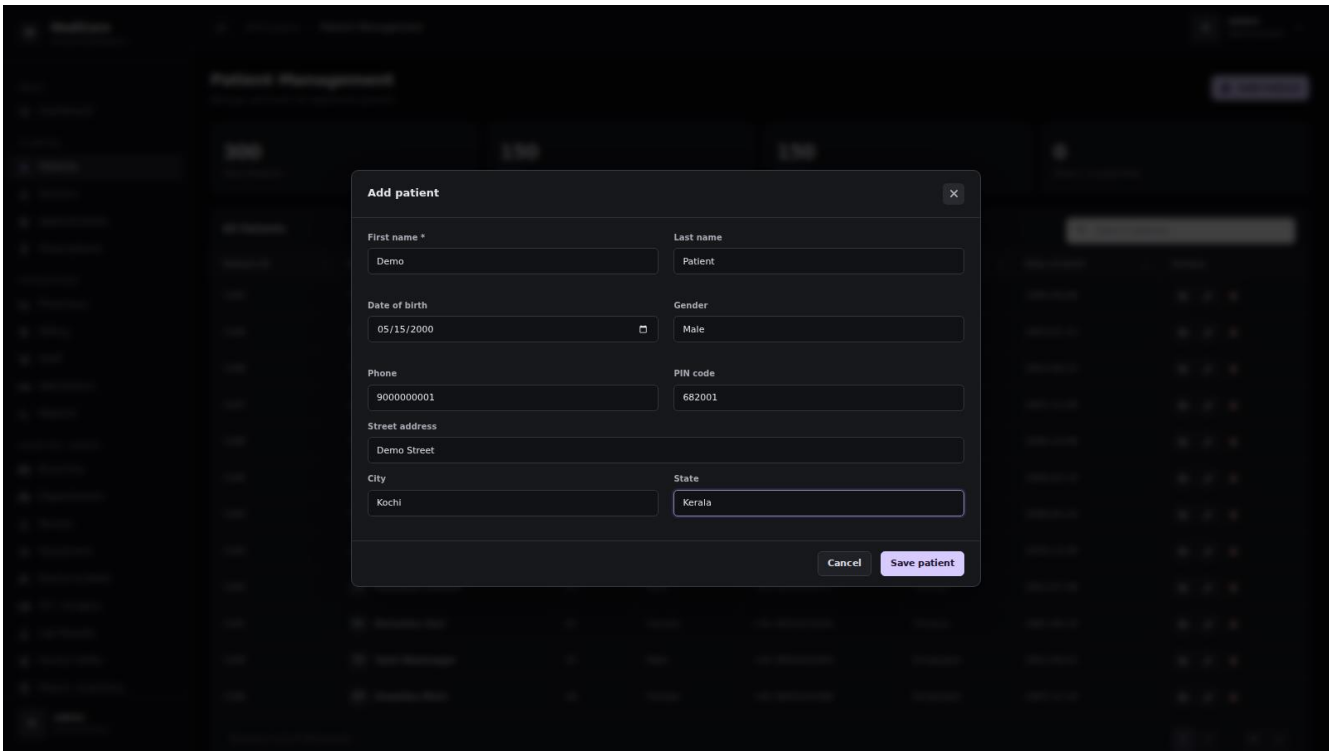


Login: user credentials open the permitted hospital workspace. All application screenshots use a 1600 × 900 desktop viewport.

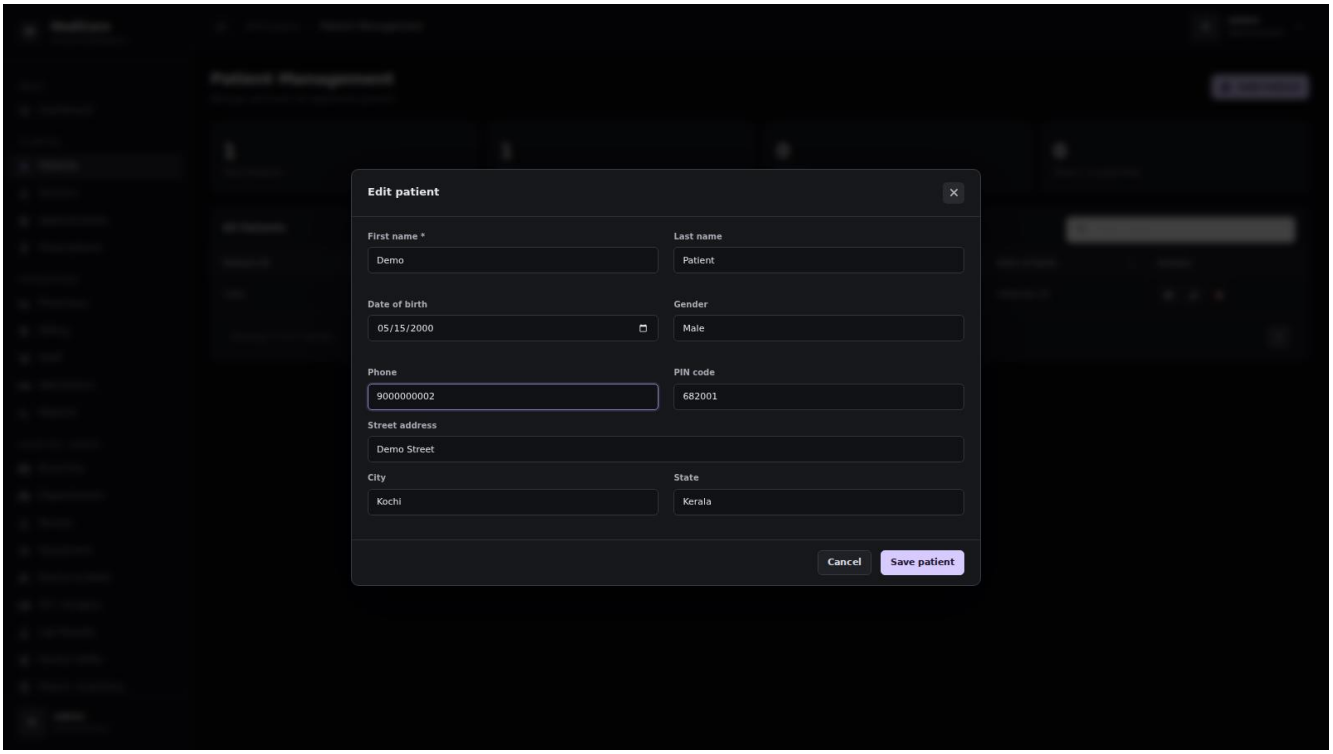


Dashboard: year selection and summary cards provide an overview of hospital activity.

4. UI Design (continued)

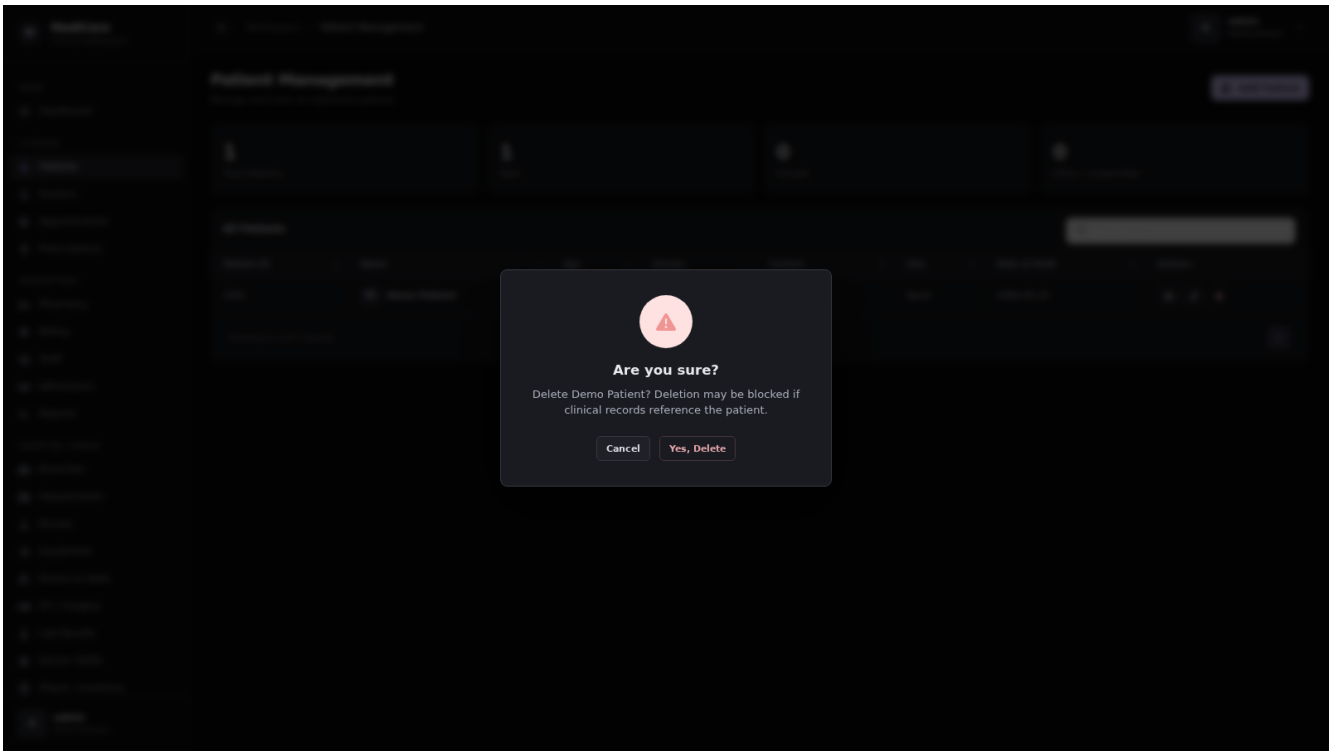


CREATE: the patient form records demographic and contact details. Saving assigns a unique PatientID.

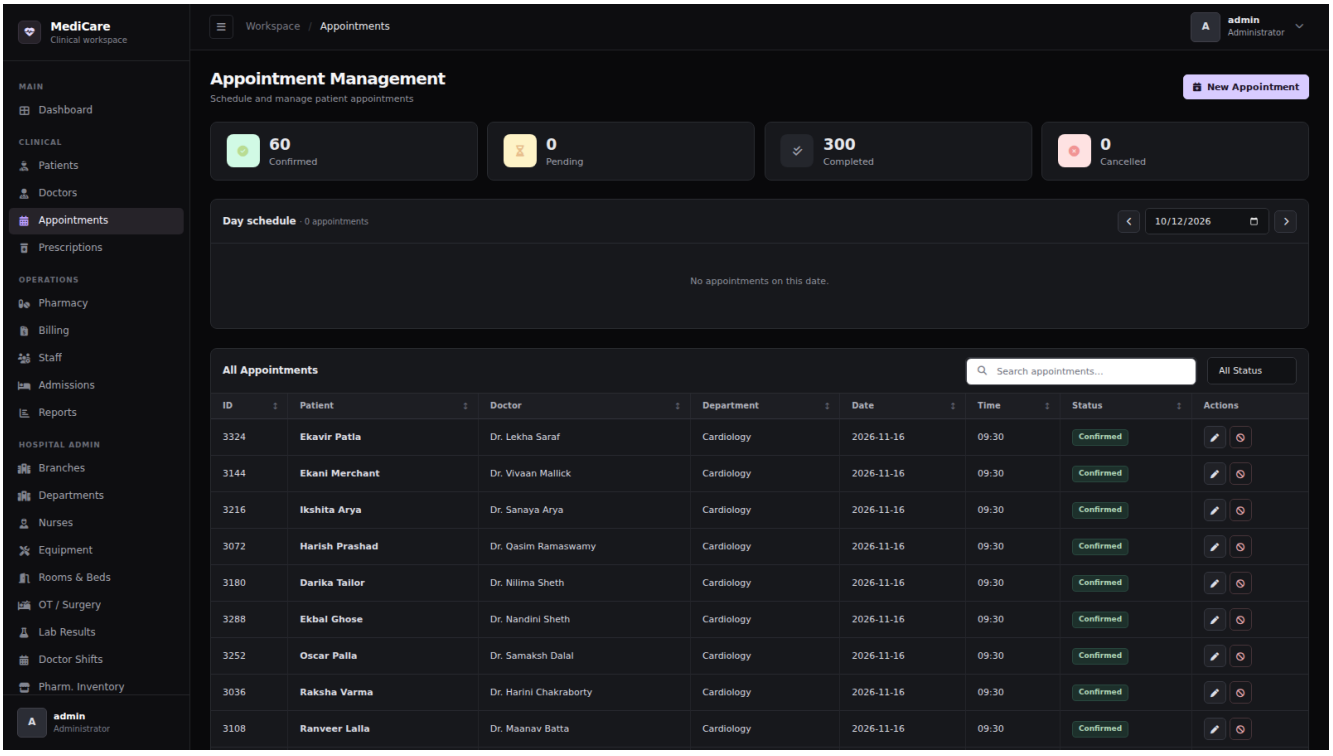


UPDATE: search locates the patient; the edit form changes contact details while retaining the same PatientID.

Data entry uses labelled fields and required values. The same register supports reading and searching before an update.



DELETE: the confirmation dialog protects against accidental removal. Referenced records remain subject to foreign-key restrictions.

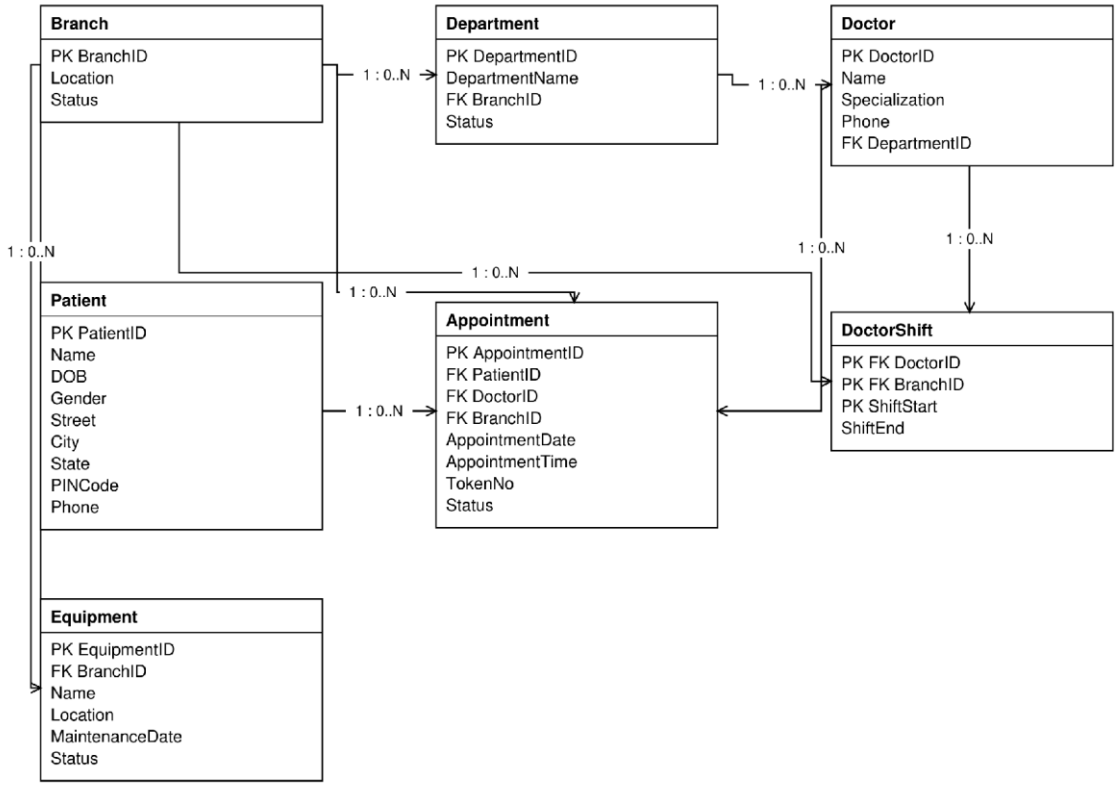


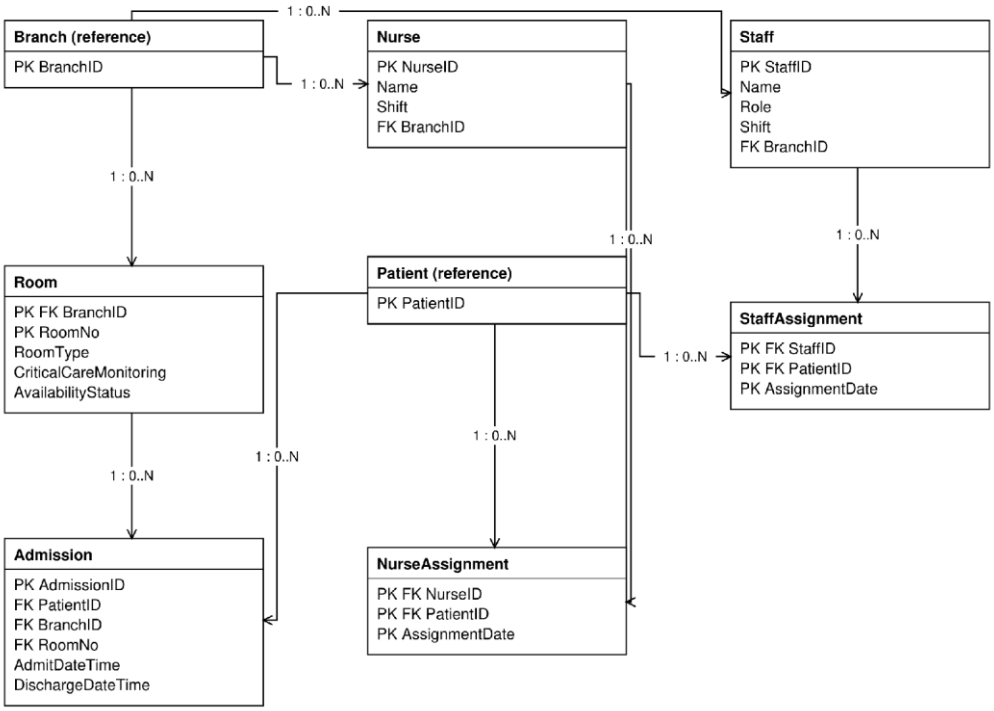
READ with JOINs: appointment rows combine patient, doctor and branch information. Their foreign keys prevent links to missing records and allow names to be displayed instead of isolated IDs.

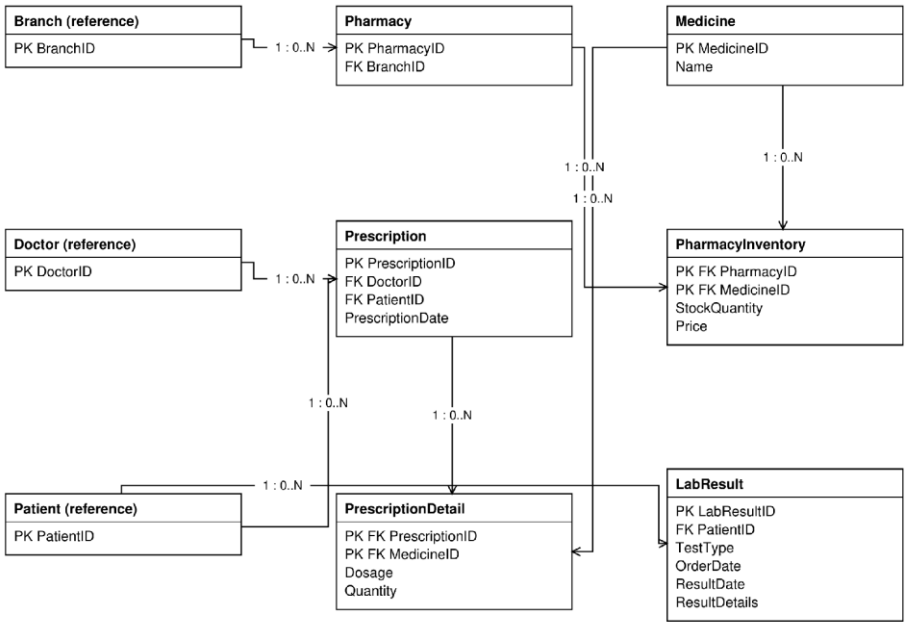
The joined appointment view connects Appointment.PatientID -> Patient, Appointment.DoctorID -> Doctor and Appointment.BranchID -> Branch.

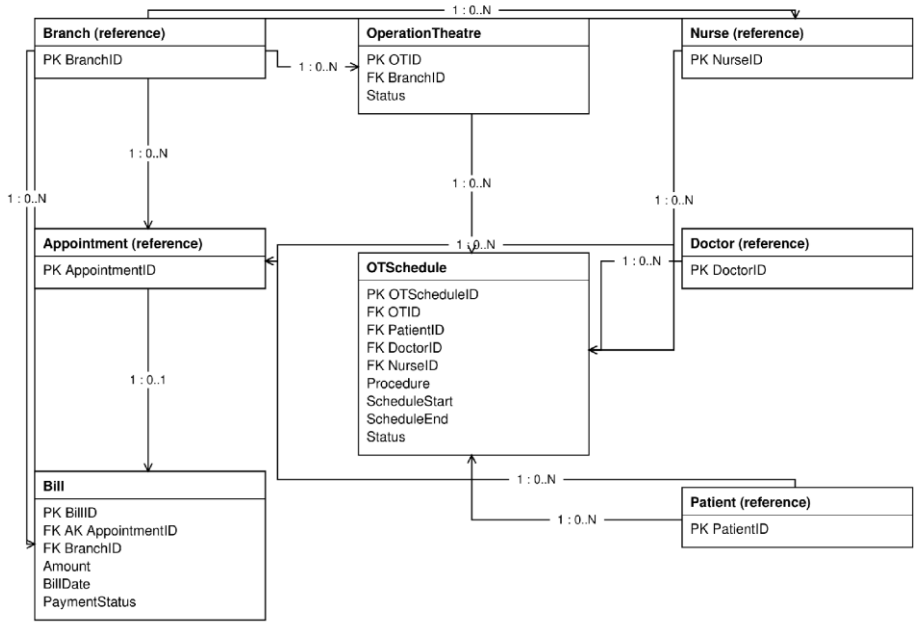
5. Database Design

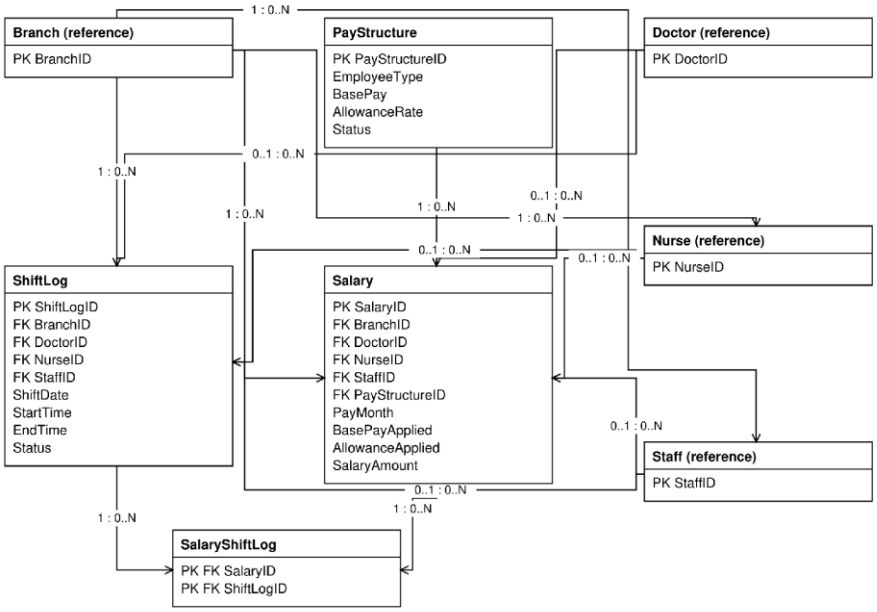
PK and FK used instead of underlines.




1 : 0..N
Branch (reference)
PK BranchID
1 : 0..N
Nurse
PK NurseID
Name
Shift
FK BranchID
Staff
PK StaffID
Name
Role
Shift
FK BranchID
1 : 0..N
1 : 0..N
1 : 0..N
Room
PK FK BranchID
PK RoomNo
RoomType
CriticalCareMonitoring
AvailabilityStatus
Patient (reference)
PK PatientID
StaffAssignment
PK FK StaffID
PK FK PatientID
PK AssignmentDate
1 : 0..N
1 : 0..N
1 : 0..N
1 : 0..N
Admission
PK AdmissionID
FK PatientID
FK BranchID
FK RoomNo
AdmitDateTime
DischargeDateTime
NurseAssignment
PK FK NurseID
PK FK PatientID
PK AssignmentDate


Branch (reference)
PK BranchID
1 : 0..N
Pharmacy
PK PharmacyID
FK BranchID
Medicine
PK MedicineID
Name
1 : 0..N
1 : 0..N
1 : 0..N
Doctor (reference)
PK DoctorID
1 : 0..N
Prescription
PK PrescriptionID
FK DoctorID
FK PatientID
PrescriptionDate
PharmacyInventory
PK FK PharmacyID
PK FK MedicineID
StockQuantity
Price
1 : 0..N
1 : 0..N
1 : 0..N
Patient (reference)
PK PatientID
PrescriptionDetail
PK FK PrescriptionID
PK FK MedicineID
Dosage
Quantity
LabResult
PK LabResultID
FK PatientID
TestType
OrderDate
ResultDate
ResultDetails


Branch (reference)
PK BranchID
OperationTheatre
PK OTID
FK BranchID
Status
Nurse (reference)
PK NurseID
Appointment (reference)
PK AppointmentID
OTSchedule
PK OTScheduleID
FK OTID
FK PatientID
FK DoctorID
FK NurseID
Procedure
ScheduleStart
ScheduleEnd
Status
Doctor (reference)
PK DoctorID
Bill
PK BillID
FK AK AppointmentID
FK BranchID
Amount
BillDate
PaymentStatus
Patient (reference)
PK PatientID
1 : 0..N
1 : 0..1


Branch (reference)
PK BranchID
PayStructure
PK PayStructureID
EmployeeType
BasePay
AllowanceRate
Status
Doctor (reference)
PK DoctorID
Nurse (reference)
PK NurseID
ShiftLog
PK ShiftLogID
FK BranchID
FK DoctorID
FK NurseID
FK StaffID
ShiftDate
StartTime
EndTime
Status
Salary
PK SalaryID
FK BranchID
FK DoctorID
FK NurseID
FK StaffID
FK PayStructureID
PayMonth
BasePayApplied
AllowanceApplied
SalaryAmount
Staff (reference)
PK StaffID
SalaryShiftLog
PK FK SalaryID
PK FK ShiftLogID
1 : 0..N
0..1 : 0..N

5.2 Entities, Attributes and Relational Schema

PK attributes are shown in bold; FK attributes are identified in the line below each relation.

Branch(BranchID, Location, Status)

FK: none.

Department(DepartmentID, DepartmentName, BranchID, Status)

FK: BranchID -> Branch.

Doctor(DoctorID, Name, Specialization, Phone, DepartmentID)

FK: DepartmentID -> Department.

DoctorShift(DoctorID, BranchID, ShiftStart, ShiftEnd)

FK: DoctorID -> Doctor; BranchID -> Branch.

Patient(PatientID, Name, DOB, Gender, Street, City, State, PINCode, Phone)

FK: none.

Appointment(AppointmentID, PatientID, DoctorID, BranchID, AppointmentDate, AppointmentTime, TokenNo, Status)

FK: PatientID -> Patient; DoctorID -> Doctor; BranchID -> Branch.

Nurse(NurseID, Name, Shift, BranchID)

FK: BranchID -> Branch.

Staff(StaffID, Name, Role, Shift, BranchID)

FK: BranchID -> Branch.

NurseAssignment(NurseID, PatientID, AssignmentDate)

FK: NurseID -> Nurse; PatientID -> Patient.

StaffAssignment(StaffID, PatientID, AssignmentDate)

FK: StaffID -> Staff; PatientID -> Patient.

Room(BranchID, RoomNo, RoomType, CriticalCareMonitoring, AvailabilityStatus)

FK: BranchID -> Branch.

Admission(AdmissionID, PatientID, BranchID, RoomNo, AdmitDateTime, DischargeDateTime)

FK: PatientID -> Patient; BranchID, RoomNo -> Room.

Prescription(PrescriptionID, DoctorID, PatientID, PrescriptionDate)

FK: DoctorID -> Doctor; PatientID -> Patient.

Medicine(MedicineID, Name)

FK: none.

PrescriptionDetail(PrescriptionID, MedicineID, Dosage, Quantity)

FK: PrescriptionID -> Prescription; MedicineID -> Medicine.

Pharmacy(PharmacyID, BranchID)

FK: BranchID -> Branch.

PharmacyInventory(PharmacyID, MedicineID, StockQuantity, Price)

FK: PharmacyID -> Pharmacy; MedicineID -> Medicine.

LabResult(LabResultID, PatientID, TestType, OrderDate, ResultDate, ResultDetails)

FK: PatientID -> Patient.

OperationTheatre(OTID, BranchID, Status)

FK: BranchID -> Branch.

OTSchedule(OTScheduleID, OTID, PatientID, DoctorID, NurseID, Procedure, ScheduleStart, ScheduleEnd, Status)

FK: OTID -> OperationTheatre; PatientID -> Patient; DoctorID -> Doctor; NurseID -> Nurse.

Equipment(EquipmentID, BranchID, Name, Location, MaintenanceDate, Status)

FK: BranchID -> Branch.

Bill(BillID, AppointmentID, BranchID, Amount, BillDate, PaymentStatus)

FK: AppointmentID -> Appointment; BranchID -> Branch. AK: AppointmentID (UNIQUE).

ShiftLog(ShiftLogID, BranchID, DoctorID, NurseID, StaffID, ShiftDate, StartTime, EndTime, Status)

FK: BranchID -> Branch; DoctorID -> Doctor; NurseID -> Nurse; StaffID -> Staff. Exactly one employee FK is non-null.

PayStructure(PayStructureID, EmployeeType, BasePay, AllowanceRate, Status)

FK: none.

Salary(SalaryID, BranchID, DoctorID, NurseID, StaffID, PayStructureID, PayMonth, BasePayApplied, AllowanceApplied, SalaryAmount)

FK: BranchID -> Branch; DoctorID -> Doctor; NurseID -> Nurse; StaffID -> Staff; PayStructureID -> PayStructure. Exactly one employee FK is non-null.

SalaryShiftLog(SalaryID, ShiftLogID)

FK: SalaryID -> Salary; ShiftLogID -> ShiftLog.

Existing physical implementation additions

Original project addition	**Purpose**
Doctor.is_active	Archive a doctor without removing referenced history.
PharmacyInventory.reorder_level	Identify stock at or below a pharmacy-specific threshold.
Bill.ot_schedule_id -> OTSchedule	Optional link to a matching completed procedure.
UserAccount(UserID, Username, PasswordHash, Role, Status)	Authentication table; UserID is PK and Username is UNIQUE.

5.3 ER-to-Relational Conversion

Example 1: Room and Admission

RoomNo is only unique within a branch. Therefore, Room uses the composite primary key (BranchID, RoomNo), with BranchID also referencing Branch. Admission becomes a separate relation with its own AdmissionID, a PatientID foreign key, the two room-key columns and admission/discharge times. The composite foreign key ensures that the selected room belongs to the specified branch; separate admission records retain past allocations.

Example 2: Prescription and Medicine

A prescription may contain several medicines, and a medicine may appear in many prescriptions. PrescriptionDetail resolves this many-to-many relationship. Its key is (PrescriptionID, MedicineID), both columns are foreign keys, and Dosage and Quantity describe that particular prescription–medicine combination.

Other mapping rule	**Application in this design**
Strong entities -> one relation each	Patient, Doctor, Branch, Department, Nurse, Staff, Medicine, Pharmacy, LabResult, Equipment, OperationTheatre and PayStructure.
One-to-many -> FK in the child	Appointment, Prescription, Admission, OTSchedule and branch-owned resources.
Many-to-many -> linking relation	DoctorShift, NurseAssignment, StaffAssignment, PharmacyInventory and SalaryShiftLog.
At most one bill per appointment	Bill stores AppointmentID as NOT NULL and UNIQUE.
Employee-specific records	ShiftLog and Salary contain three optional employee FKs with an exactly-one rule.

5.4 Functional Dependencies

For every single-ID entity, its primary key determines all other attributes. Example: PatientID -> Name, DOB, Gender, Street, City, State, PINCode, Phone. DoctorID -> Name, Specialization, Phone, DepartmentID. The same rule applies to Branch, Department, Appointment, Nurse, Staff, Admission, Prescription, Medicine, Pharmacy, LabResult, OperationTheatre, OTSchedule, Equipment, ShiftLog, PayStructure and Salary.

Relation	**Key dependency**
Room	(BranchID, RoomNo) -> RoomType, CriticalCareMonitoring, AvailabilityStatus
DoctorShift	(DoctorID, BranchID, ShiftStart) -> ShiftEnd
PrescriptionDetail	(PrescriptionID, MedicineID) -> Dosage, Quantity
PharmacyInventory	(PharmacyID, MedicineID) -> StockQuantity, Price
NurseAssignment / StaffAssignment	(NurseID or StaffID, PatientID, AssignmentDate) identifies the complete assignment.
SalaryShiftLog	(SalaryID, ShiftLogID) is the complete relation.
Bill	BillID -> all attributes; AppointmentID -> all attributes (alternate key).

5.5 Normalization

Normalization separates reusable master data from transactions and junction records. The following examples explain the process; the same principles are applied across the remaining relations. [2]

Example 1: Pharmacy stock

Raw record: PharmacyID, BranchID, MedicineID,
MedicineName, StockQuantity, Price
PharmacyID -> BranchID
MedicineID -> MedicineName
(PharmacyID, MedicineID) -> StockQuantity, Price

Keeping all these fields in one stock table repeats a medicine name at every pharmacy and a branch identifier for every stock item. Separating Pharmacy, Medicine and PharmacyInventory removes these partial dependencies. A medicine name is corrected once in Medicine; local stock and price remain in PharmacyInventory.

Example 2: Doctor and Department

DoctorID -> DepartmentID
DepartmentID -> DepartmentName, BranchID, Status
Doctor(DoctorID, Name, Specialization, Phone, DepartmentID)
Department(DepartmentID, DepartmentName, BranchID, Status)

Normal form	Application to the remaining design
1NF	Each field holds one value. Patient addresses use separate Street, City, State and PINCode fields. Multiple medicines are represented by separate PrescriptionDetail rows.
2NF	Room, DoctorShift, PrescriptionDetail and PharmacyInventory keep attributes dependent on the whole composite key. Care assignments and SalaryShiftLog do not repeat descriptive master data.
3NF	Branch locations, department descriptions and medicine names remain in their own master tables, avoiding non-key descriptive dependencies in transaction records.
BCNF assessment	Under the listed logical dependencies, key determinants satisfy the BCNF criterion; Bill's alternate determinant AppointmentID is itself a candidate key. Additional business dependencies must also be considered.

5.6 Constraints and Cardinalities

Rule	Implementation and purpose
Identity and mandatory links	PRIMARY KEY and NOT NULL identify records; FOREIGN KEY prevents references to missing parents.
Composite identity	Room: (BranchID, RoomNo). PrescriptionDetail, PharmacyInventory and assignment tables use combined keys.
Unique values	Department name is unique within its branch; a branch/date/token identifies one appointment; Bill.AppointmentID is unique.
Valid domains	CHECK constraints restrict branch, appointment, room, theatre, equipment, billing and payroll statuses.
Numeric values	Tokens and prescription quantities are positive. Stock, prices, bill amounts and pay amounts cannot be negative.
Valid time order	Shift and procedure end times must follow start times. Discharge follows admission; laboratory result date cannot precede order date.
Scheduling integrity	Original SQL prevents defined overlaps in doctor shifts, active appointments, admissions and theatre schedules. Appointment checks also require a valid branch shift.
Employee and payroll links	Exactly one employee type is selected; the pay structure must match. Linked shift logs must match the employee, branch, month and completed status.
Billing integrity	The API requires a completed appointment. SQL checks branch agreement and any linked completed OT schedule.
Relationship cardinality	Most parent-child links are 1 : 0..N; Appointment -> Bill is 1 : 0..1. Junction tables implement many-to-many relationships.

6. Database Selection & Justification

PostgreSQL is selected because it supports the relational structure and integrity requirements of hospital records. Primary and foreign keys, composite keys, CHECK constraints, transactions, indexes and SQL JOINs support reliable storage and retrieval. Its time-range and overlap facilities are useful for appointments, rooms and theatre schedules. PostgreSQL is open source, has no database licence fee, and can support growth beyond the demonstration dataset. pgAdmin 4 provides a graphical interface to inspect tables, execute SQL in the Query Tool and view outputs. It is the administration tool; PostgreSQL is the DBMS that stores and processes the data.

7. Implementation

7.1 Database setup and data loading

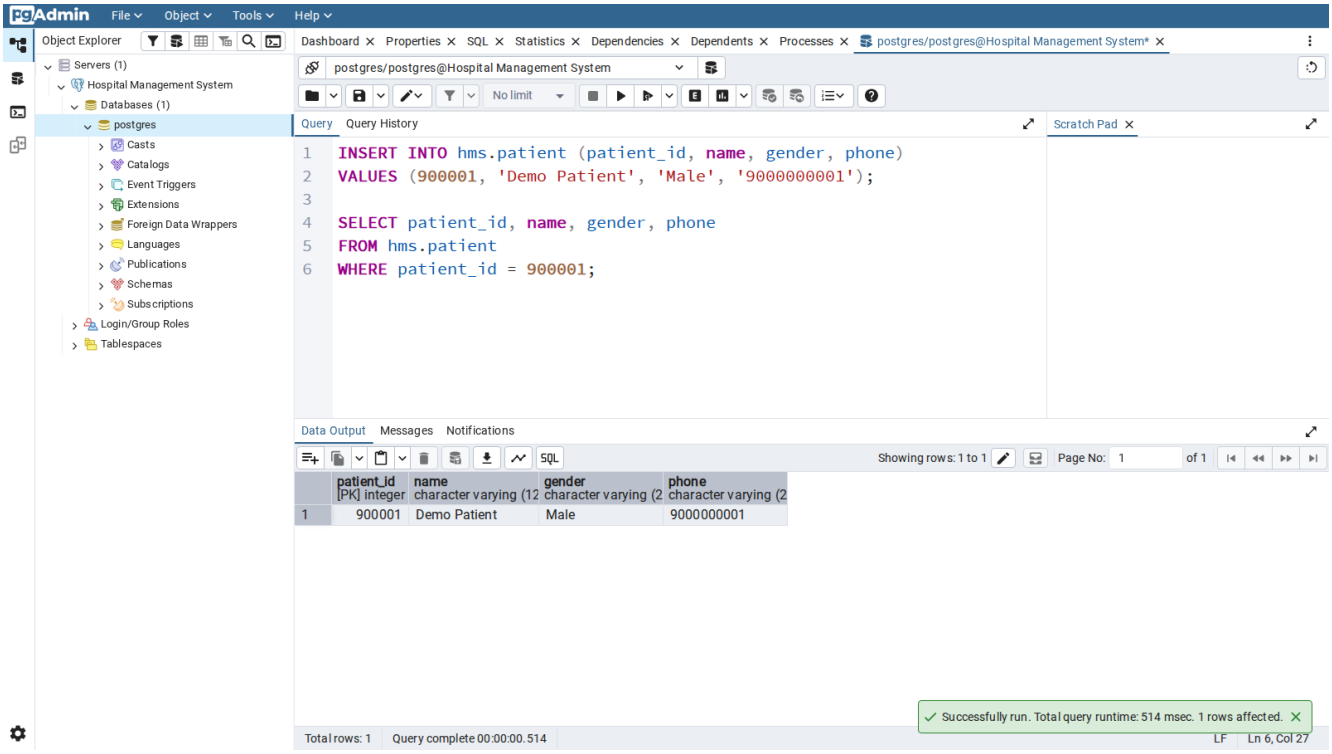
The database is created from the command line before the application starts. The supplied SQL files then create the hms schema, tables, keys and validation constraints. Raw CSV records are cleaned, separated into related records and loaded into the database. The browser interface communicates with an Express API, which reads and writes PostgreSQL records.

createdb -U postgres hms
psql -U postgres -d hms -f database/sql/SCHEMA.sql
psql -U postgres -d hms -f database/sql/ADDITIONAL.sql

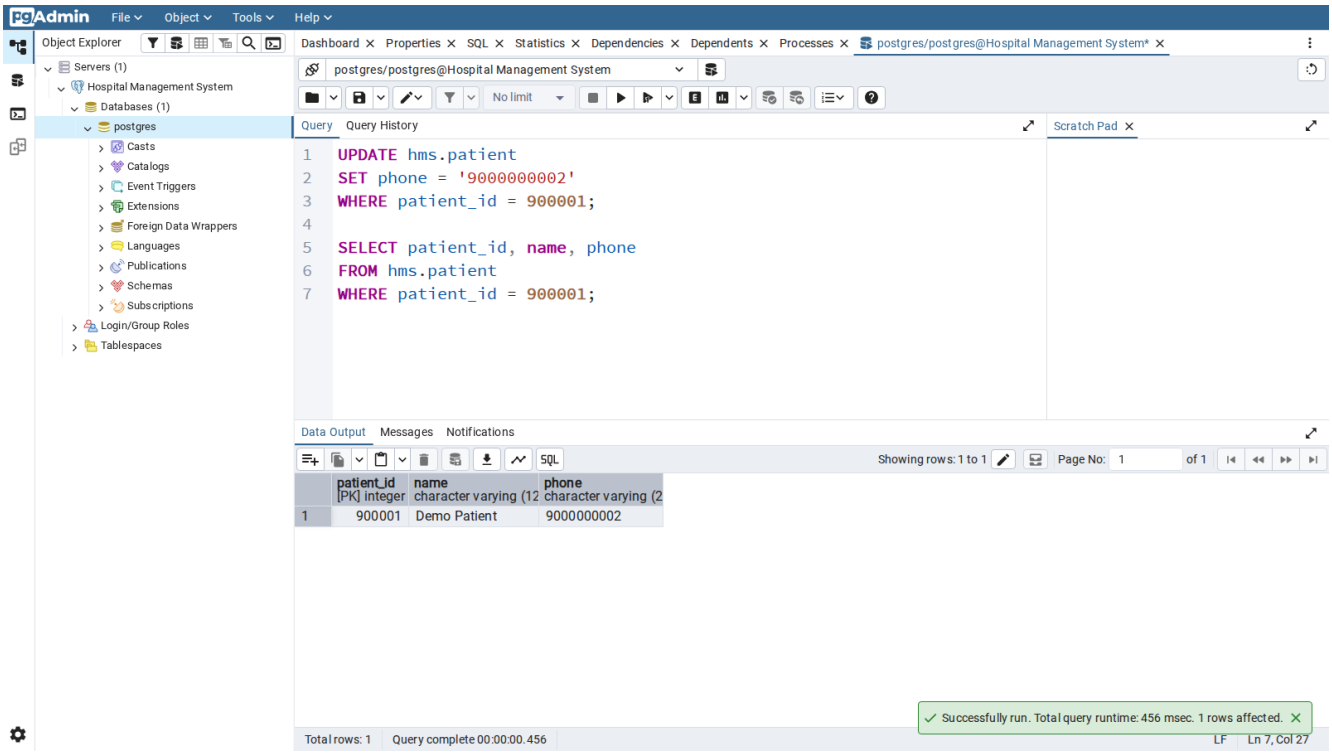
Example table definition:

```
CREATE TABLE medicine (
   medicine_id INTEGER GENERATED BY DEFAULT AS IDENTITY
      PRIMARY KEY,
   name VARCHAR(150) NOT NULL
);
```

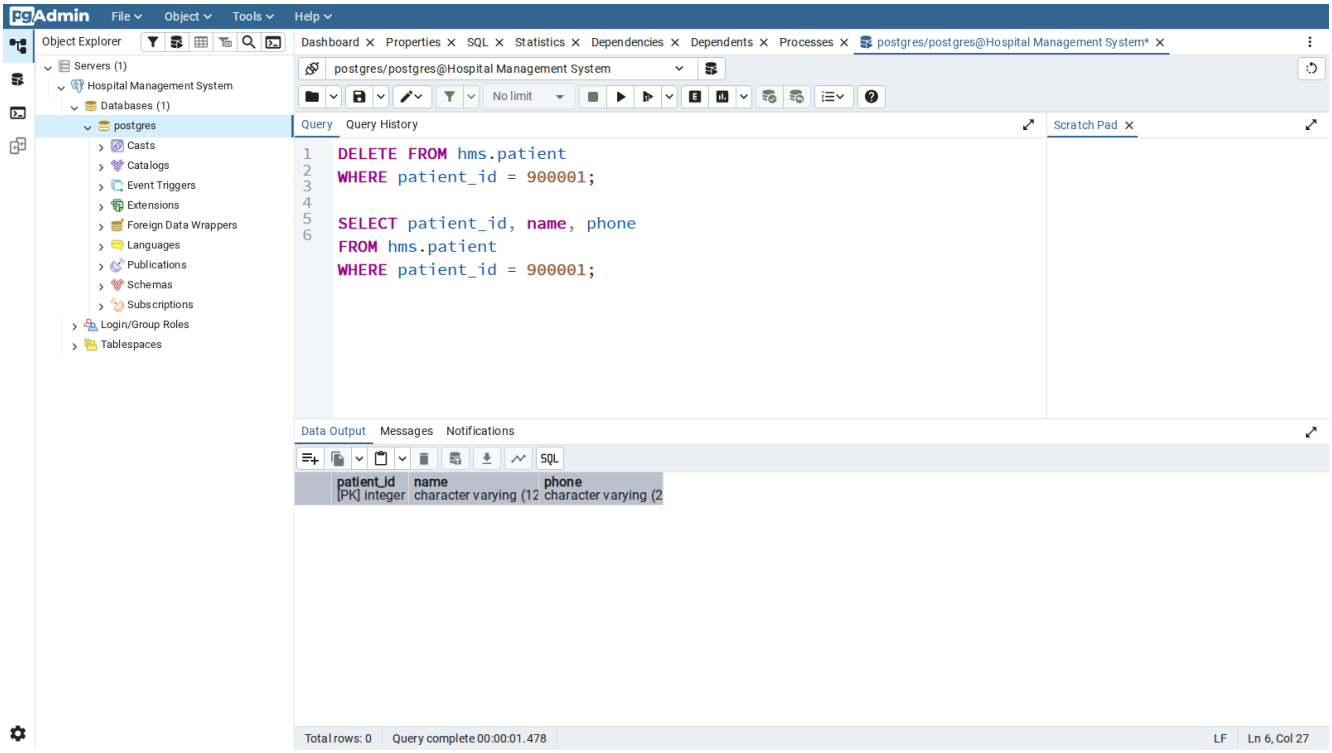
7.2 INSERT: create a patient record



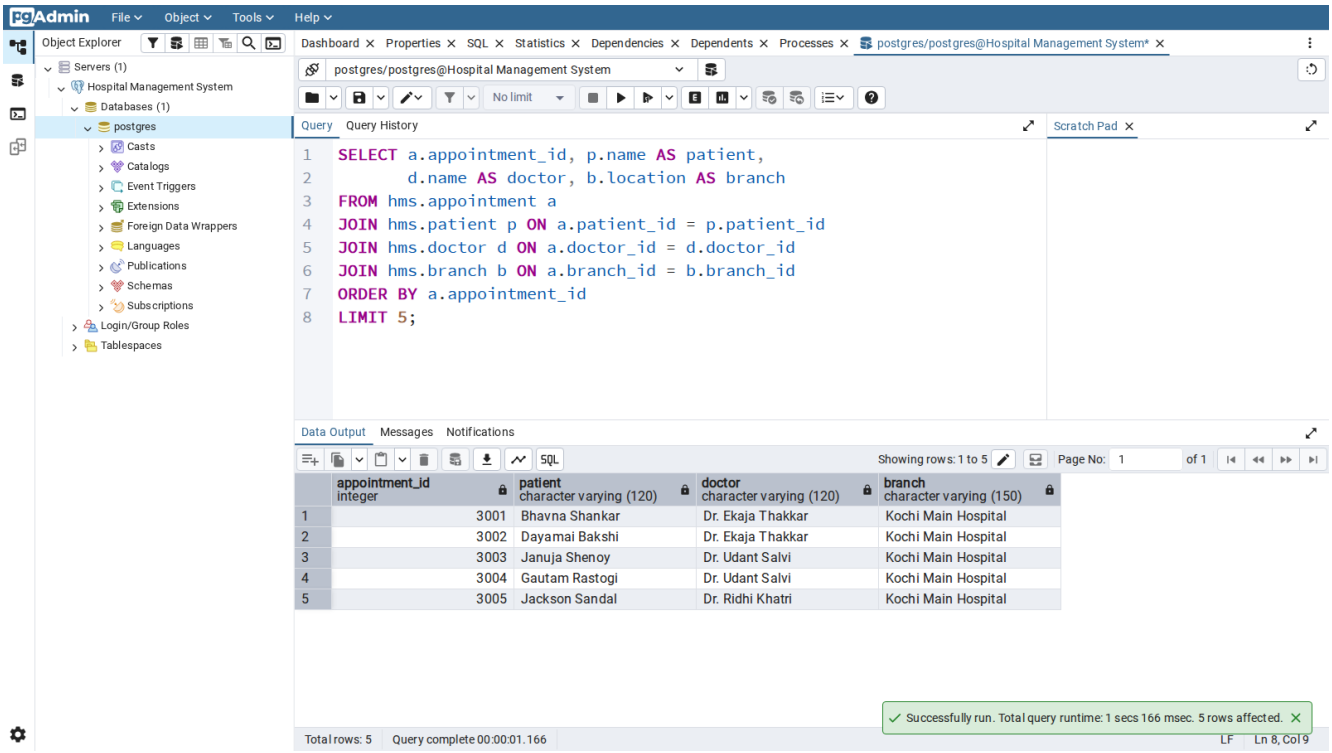
INSERT creates the demonstration patient; SELECT displays the stored row.



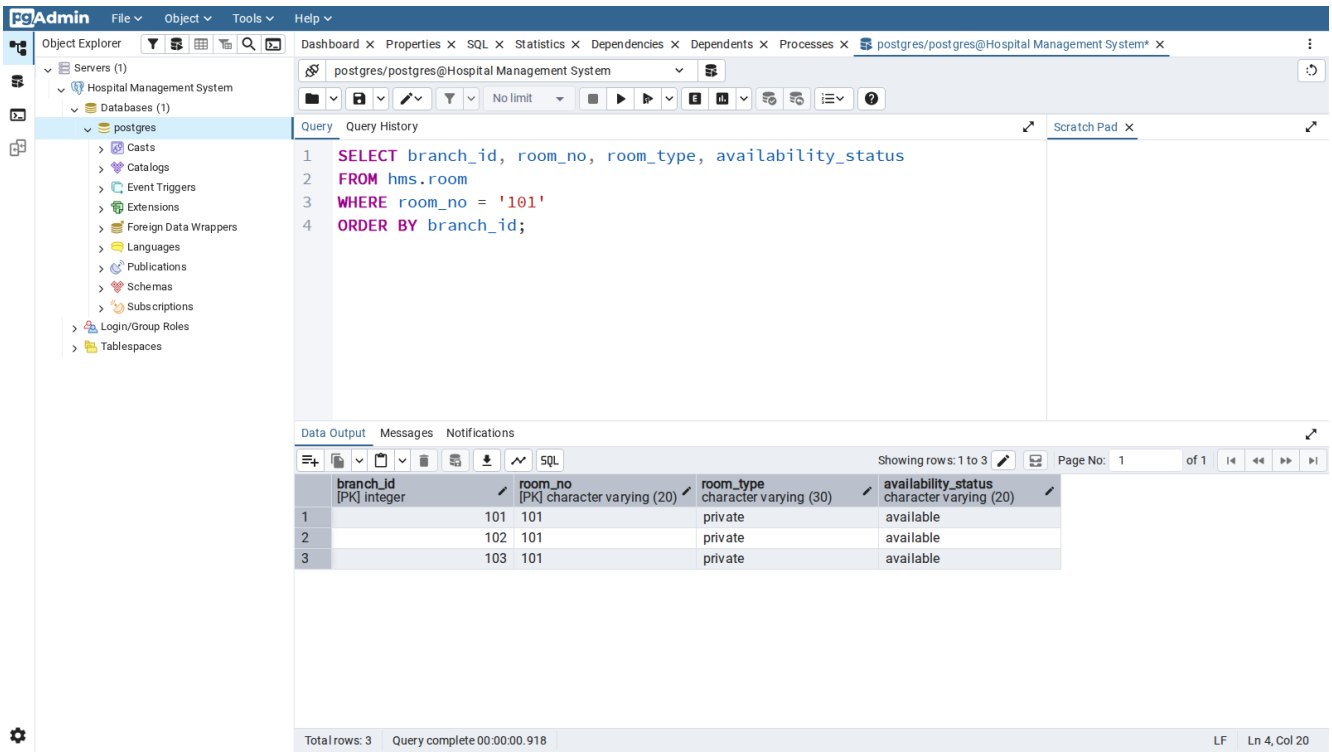
UPDATE: change the patient's phone number. The following SELECT confirms the new value.



DELETE: remove the unreferenced demonstration patient. The following SELECT returns no rows.



SELECT with JOINs: foreign-key columns connect Appointment to Patient, Doctor and Branch, producing readable appointment rows.



Database table output: Room uses BranchID and RoomNo together, so the same room number can be reused in another branch.

Additional strengths: transactions protect related writes, pharmacy stock is checked before dispensing, and linked identifiers make cross-module reporting possible.

8. Testing & Validation

The original SQL files, original API and original UI were exercised using an isolated demonstration database. The checks below record observed outcomes; a passing sample does not mean every workflow or production condition has been tested.

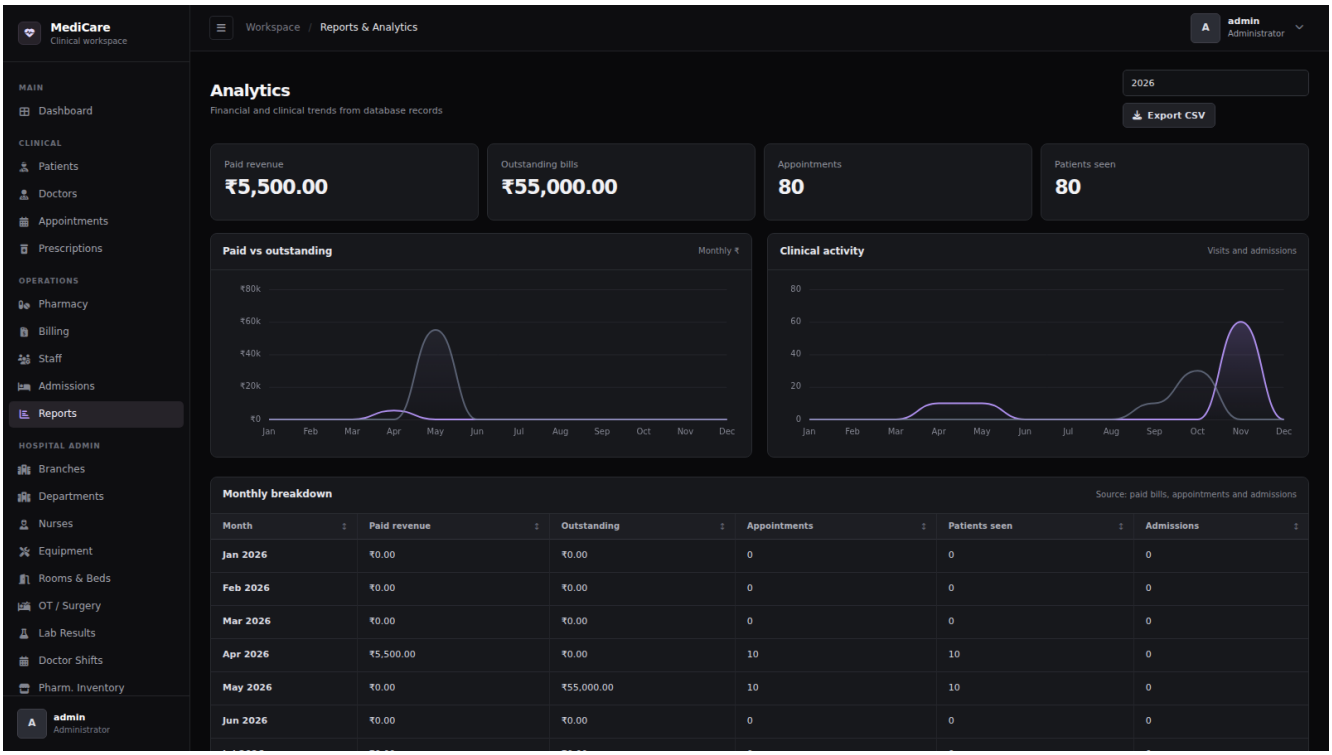
Test	Input / action	Expected output	Actual output	Result
Patient creation	Valid demographic form	Record saved	HTTP 201; record found	Pass
Search/read	Search “Demo Patient”	Matching row displayed	Matching record displayed	Pass
Patient update	Change phone to 9000000002	Same ID; new phone	HTTP 200; value confirmed	Pass
Patient deletion	Delete unreferenced test patient	Record removed	HTTP 200; row count 0	Pass
Missing parent	Lab result with unknown PatientID	Reject FK violation	SQLSTATE 23503	Pass
Duplicate key	Insert existing BranchID 101	Reject duplicate	SQLSTATE 23505	Pass
Invalid quantity	Set stock to -1	Reject CHECK violation	SQLSTATE 23514	Pass
Duplicate bill	Reuse billed AppointmentID	Reject duplicate	SQLSTATE 23505	Pass
Overlapping admission	Repeat room/time allocation	Reject overlap	SQLSTATE 23P01	Pass
Report export	Export year 2024	12 monthly rows + header	13 CSV lines	Pass
Included unit suite	Run two original controller test files	All included assertions pass	23 passed; 0 failed	Pass

Validation coverage

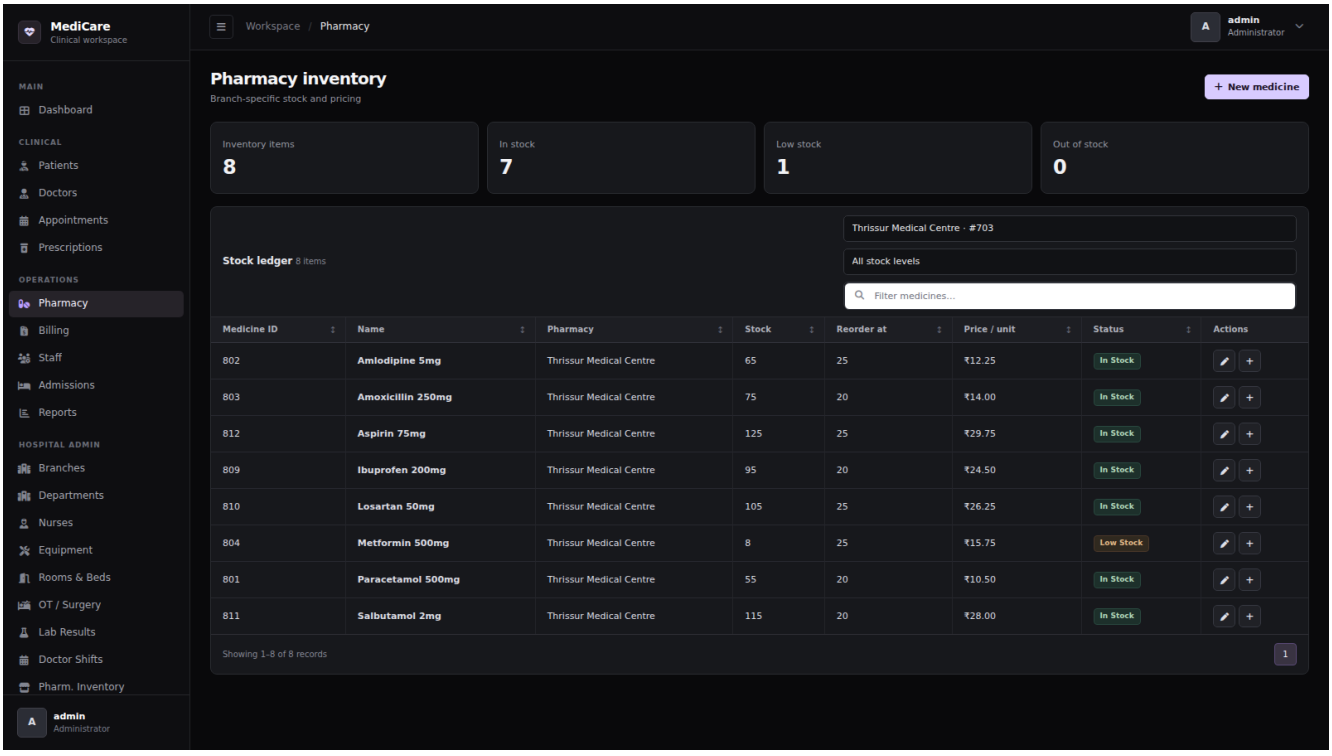
- Primary-key and unique checks prevent duplicate identities and duplicate appointment bills.
- The foreign-key test demonstrates that a clinical result cannot refer to a missing patient.
- CHECK and overlap tests demonstrate that invalid quantities and conflicting allocations are rejected.
- CREATE, UPDATE and DELETE were verified against stored data, not only success messages.

Limitations observed: a login-page password-event error remains in the original UI, although login succeeds. Broader unresolved items are listed separately in help.md. Performance, recovery, deployment and clinical safety were not certified by these tests.

9. Results & Screenshots



Year-based reporting: hospital activity and billing summaries can be reviewed and exported. “Outstanding bills” represents the face value of unpaid/partial bills, not a payment-ledger balance.



Pharmacy register: stock, prices and reorder thresholds are maintained per pharmacy. Low-stock rows support replenishment decisions.

10. Challenges Faced & Solutions

Challenge	Solution demonstrated in the original project
Repeated data in separate registers	Separate master entities and transaction tables; connect them with identifiers and foreign keys.
Room numbers repeated across branches	Use (BranchID, RoomNo) as a composite key and reference both columns from Admission.
Several medicines on one prescription	Use PrescriptionDetail to store each medicine and its dosage/quantity separately.
Competing schedules	Use shift checks and overlap constraints for the defined consultation, admission and theatre time ranges.
Stock consistency	Keep stock per pharmacy and validate available quantity before reducing it.
Combining information for the UI	Use JOINs to display patient, doctor, branch and medicine details with their transaction records.
Connecting CSV records to relational tables	Clean and map raw values before loading parent records and related child records.

11. Conclusion & Future Enhancements

The project achieves its core academic objective of organizing hospital records into a connected relational database and exposing useful record-management operations through a web interface. It demonstrates a practical transition from separate paper-style registers to digital patient, clinical and administrative records. The completed work supports consistent retrieval, updating and reporting across branches.

Major strengths

- A broad, connected design with 26 core relations covering clinical, pharmacy, billing and payroll activities.
- Composite keys and junction tables model branch-specific rooms, medicine details and many-to-many relationships correctly.
- Foreign keys, value checks and scheduling constraints protect important relationships and business rules.
- Searchable CRUD screens, readable JOIN results and year-based CSV reports make database concepts visible in the application.

Future enhancements

Strengthen security and audit logging; resolve the documented original-version issues; add payment and dispensing histories; optimize frequently used queries; provide patient-facing web/mobile access; and package deployment with Docker and reliable backup procedures. These are proposed extensions, not completed features.

References

- draw.io, diagram editor used for the editable ER model. https://app.diagrams.net/
- pgAdmin4 and postgreSQL 17 related resources.
- Faker for producing dummy values to fill database.